BARQ ACADEMY

DevOps Internship - Kubernetes Cluster

Kubernetes Cluster Deployment Guide & Evidence

INTERN NAME	Zinab Mohamed Elsayed
Task	Kubernetes Cluster

Table of Contents

Introduction & Overview 3
Environment 3
VM Preparation 3
1. Prerequisites & System Configuration (Both Nodes) 4
1.1 Disable swap 4
1.2 Load kernel modules and set sysctl 4
1.3 Set SELinux to permissive and update the system 5
2. Installing Container Runtime & Kubernetes Tools (Both Nodes) 5
2.1 Install containerd 5
2.2 Add the Kubernetes yum repository (required on Rocky 9) 6
2.3 Install kubelet, kubeadm, kubectl 6
3. Initializing the Master Node 6
4. Installing the CNI Network Plugin (Flannel) 7
5. Joining the Worker Node 7
6. Verifying Node Readiness 8
7. Managing Control Plane Taints 8
8. Deploying the Multi-Container Application 9
9. Services & Endpoints Verification 10
10. Connectivity & Response Testing 11
10.1 Test the app ClusterIP service from inside the cluster 11
10.2 Test health, readiness, records and counter 11
10.3 Test NGINX from outside the cluster (host) 12
10.4 Cross-node pod networking test (`network-test.yaml`) 12
11. BONUS: CI/CD Pipeline (Build, Push, Deploy with GitHub Actions) 13
11.1 How things worked BEFORE the bonus (manual flow) 13
11.2 Exact steps done for the bonus (with the actual commands) 14
11.3 Result of the bonus 16
Troubleshooting Quick Reference 17

Introduction & Overview

This document provides a comprehensive, step-by-step technical walkthrough of setting up a 2-node Kubernetes cluster (1 Master, 1 Worker) on Rocky Linux 9 from scratch, configuring networking, managing cluster taints, and deploying the multi-container application (PostgreSQL + Redis + Python app replicas + NGINX) into the `barq-assessment` namespace.

Environment

Role	Hostname	OS	CPU	RAM	Disk	IP
Master	k8s-master	Rocky Linux 9	2+	4 GB	20 GB+	192.168.66.132
Worker	k8s-worker1	Rocky Linux 9 (full clone of master **before** `kubeadm init`)	2	2 GB	20 GB+	192.168.66.133

Cluster parameters:

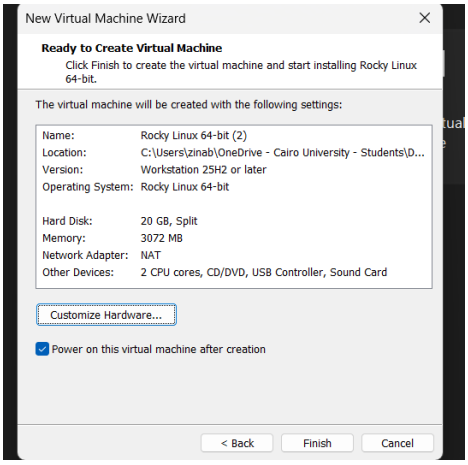
- Pod network CIDR: `10.244.0.0/16` (Flannel default)
- CNI: Flannel
- Container runtime: containerd (with `SystemdCgroup = true`)
- Namespace: `barq-assessment`
- Application manifests (in this repo under `k8s/`): `namespace.yaml`, `secret.yaml`,

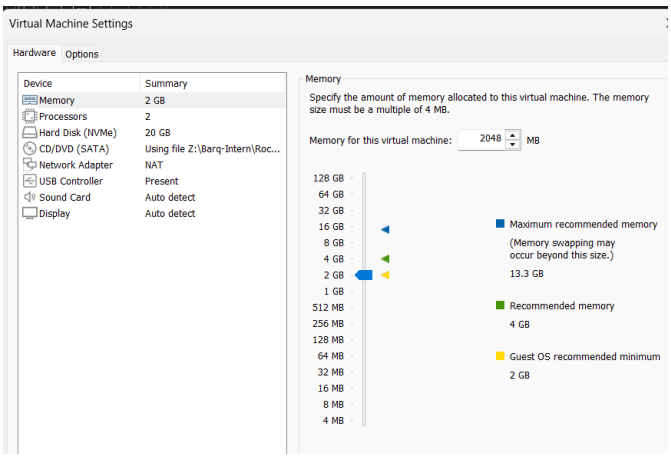
`postgres.yaml`, `redis.yaml`, `app.yaml`, `nginx.yaml`

VM Preparation

1. Install Rocky Linux 9 on the master VM (user `zinabelamir`, or your own user).
2. Create the worker as a **full clone** of the master **before** running `kubeadm init`:
   In VMWare : right-click master name -> Manage -> Clone -> Full clone.
3. Set worker resources to **2 GB RAM** (minimum) and 2 vCPUs.

Make the master with this configurations: , worker :





Power on both VMs and confirm they can reach each other and the internet:

```
ip addr
ping -c 3 192.168.66.133 (for master)
ping -c 3 192.168.66.132 (for worker)
```

```
[zinabelamir@k8s-master ~]$ ping -c 4 192.168.66.133
PING 192.168.66.133 (192.168.66.133) 56(84) bytes of data.
64 bytes from 192.168.66.133: icmp_seq=1 ttl=64 time=5.45 ms
64 bytes from 192.168.66.133: icmp_seq=2 ttl=64 time=0.362 ms
64 bytes from 192.168.66.133: icmp_seq=3 ttl=64 time=0.744 ms
64 bytes from 192.168.66.133: icmp_seq=4 ttl=64 time=0.440 ms

--- 192.168.66.133 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3040ms
rtt min/avg/max/mdev = 0.362/1.748/5.449/2.141 ms
```

```
[zinabelamir@k8s-worker1 ~]$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens160: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0c:29:4b:b8:53 brd ff:ff:ff:ff:ff:ff
    altname enp3s0
    inet 192.168.66.133/24 brd 192.168.66.255 scope global dynamic noprefixroute ens160
       valid_lft 1114sec preferred_lft 1114sec
    inet6 fe80::20c:29ff:fe4b:b853/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
[zinabelamir@k8s-worker1 ~]$ ping -c 4 192.168.66.132
PING 192.168.66.132 (192.168.66.132) 56(84) bytes of data.
64 bytes from 192.168.66.132: icmp_seq=1 ttl=64 time=11.8 ms
64 bytes from 192.168.66.132: icmp_seq=2 ttl=64 time=3.87 ms
64 bytes from 192.168.66.132: icmp_seq=3 ttl=64 time=0.509 ms
64 bytes from 192.168.66.132: icmp_seq=4 ttl=64 time=0.359 ms
```

1. Prerequisites & System Configuration (Both Nodes)

Run on **both** master and worker before initializing Kubernetes.

1.1 Disable swap

Kubernetes requires swap to be off.

```
sudo swapoff -a
sudo sed -i.bak '/\sswap\s/s/^/#/' /etc/fstab
free -h
```

Expected: the `Swap:` line shows `0B` used.

```
[zinabelamir@k8s-master ~]$ sudo swapoff -a
[sudo] password for zinabelamir:
[zinabelamir@k8s-master ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           2.7Gi       1.3Gi       1.0Gi        23Mi       570Mi       1.4Gi
Swap:             0B          0B          0B
[zinabelamir@k8s-master ~]$ ^C
[zinabelamir@k8s-master ~]$ sudo sed -i.bak '/\sswap\s/s/^/#/' /etc/fstab
```

1.2 Load kernel modules and set sysctl

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

```
sudo tee /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
```

Verify:

```
lsmod | grep -E 'overlay|br_netfilter'
sysctl net.bridge.bridge-nf-call-iptables net.ipv4.ip_forward
```

```
[zinabelamir@k8s-master barq-task]$ lsmod | grep -E 'overlay|br_netfilter'
sysctl net.bridge.bridge-nf-call-iptables net.ipv4.ip_forward
br_netfilter           36864  0
bridge                421888  1 br_netfilter
overlay               237568  18
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
[zinabelamir@k8s-master barq-task]$
```

1.3 Set SELinux to permissive and update the system

```
sudo setenforce 0
sudo sed -i 's/^SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config
getenforce
sudo dnf update -y
```

Expected `getenforce` output: `Permissive`.

```
[zinabelamir@k8s-master barq-task]$ getenforce
Permissive
[zinabelamir@k8s-master barq-task]$
```

2. Installing Container Runtime & Kubernetes Tools (Both Nodes)

2.1 Install containerd

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y containerd.io

sudo mkdir -p /etc/containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
grep SystemdCgroup /etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl enable containerd
```

Expected: `SystemdCgroup = true` and containerd `active (running)`.

2.2 Add the Kubernetes yum repository (required on Rocky 9)

The `kubelet`/`kubeadm`/`kubectl` packages are not in the base repos, so add the upstream repository first:

```
sudo tee /etc/yum.repos.d/kubernetes.repo <<EOF
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.30/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.30/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
```

2.3 Install kubelet, kubeadm, kubectl

```
sudo dnf install -y kubelet kubeadm kubectl --disableexcludes=kubernetes
sudo systemctl enable --now kubelet
kubectl version --client
```

3. Initializing the Master Node

Run **only on `k8s-master`**:

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 -- apiserver-advertise-
address=192.168.66.132
```

Save the `kubeadm join ... --token ... --discovery-token-ca-cert-hash ...` line from the output. If you lose it later, regenerate with:

```
kubeadm token create --print-join-command
```

Configure `kubectl` for your regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

```
To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.66.132:6443 --token 20gbmo.xtpeqk0e2noj30xv \
        --discovery-token-ca-cert-hash sha256:2a2ad4fa9405e1031cbb0f96771df174058b572efa6cfc84b4ce29deafcd2785
[zinabelamir@k8s-master ~]$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
[sudo] password for zinabelamir:
```

4. Installing the CNI Network Plugin (Flannel)

Run **on the master**:

```
kubectl apply -f https://raw.githubusercontent.com/flannel-
io/flannel/master/Documentation/kube-flannel.yml
kubectl get pods -n kube-system -w
kubectl get nodes
```

Expected: Flannel pods `Running`, master node becomes `Ready`.

```
[zinabelamir@k8s-master ~]$ kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-fl
annel.yml
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
[zinabelamir@k8s-master ~]$
```

5. Joining the Worker Node

Run **on `k8s-worker1`** :

```
sudo kubeadm join 192.168.66.132:6443 --token <XXX> --discovery-token-ca-cert-hash <XXX>
```

```
}[zinabelamir@k8s-worker1 ~]$nc -zv 192.168.66.132 6443
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Connected to 192.168.66.132:6443.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
[zinabelamir@k8s-worker1 ~]$ sudo kubeadm join 192.168.66.132:6443 --token 20gbmo.xtpeqk0e2noj30xv      --discovery-token-
ca-cert-hash sha256:2a2ad4fa9405e1031cbb0f96771df174058b572efa6cfc84b4ce29deafcd2785
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 505.055849ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

[zinabelamir@k8s-worker1 ~]$
```

If the join fails because of firewall rules, open the required ports or stop firewalld for lab use:

```
sudo systemctl stop firewalld
sudo systemctl disable firewalld
```

Then verify **on the master**:

```
kubectl get nodes -o wide
```

6. Verifying Node Readiness

```
kubectl get nodes
kubectl get pods -A -o wide
```

Expected: `k8s-master` and `k8s-worker1` both `Ready`, all `kube-system` pods `Running`.

```
[zinabelamir@k8s-master ~]$ kubectl get nodes
NAME          STATUS   ROLES           AGE   VERSION
k8s-master    Ready    control-plane   40m   v1.30.14
k8s-worker1   Ready    <none>          79s   v1.30.14
```

7. Managing Control Plane Taints

By default the control plane has a `NoSchedule` taint so workloads land on the worker. Re-apply it explicitly (idempotent) and verify:

```
kubectl taint nodes k8s-master node-role.kubernetes.io/control-plane:NoSchedule
kubectl describe node k8s-master | grep -i taints
```

Expected: `Taints: node-role.kubernetes.io/control-plane:NoSchedule`.

```
[zinabelamir@k8s-master barq-task]$ kubectl describe node k8s-master | grep -i taints
Taints:             node-role.kubernetes.io/control-plane:NoSchedule
[zinabelamir@k8s-master barq-task]$
```

To allow scheduling on the master again (optional, not used in this task):

```
kubectl taint nodes k8s-master node-role.kubernetes.io/control-plane:NoSchedule-
```

8. Deploying the Multi-Container Application

The application stack is: PostgreSQL (1 replica) + Redis (1 replica) + Python `barq-app` (3 replicas) + NGINX (2 replicas, NodePort `30080`), all in namespace `barq-assessment`. The manifests live in `k8s/` in this repository.

Apply them **in order** (secret before postgres/app):

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/postgres.yaml
kubectl apply -f k8s/redis.yaml
kubectl apply -f k8s/app.yaml
kubectl apply -f k8s/nginx.yaml

kubectl -n barq-assessment get pods -o wide -w
```

Notes:

- k8s/app.yaml uses image: `localhost/barq-app:1.0` with imagePullPolicy: Never — not docker.io/zinabelamir/barq-app:latest. The image was built on the master with `podman build -t barq-app:1.0 .` (podman 5.8.2, Docker-compatible on Rocky 9.8), verified with podman run (GET / → 200, GET /health → 200), then transferred to the cluster runtime with `podman save -o /tmp/barq.tar` + `sudo ctr -n k8s.io images import` and verified with `sudo crictl images | grep barq` (localhost/barq-app 1.0, 164MB). The same import was repeated on k8s-worker1 via scp, because replicas: 3 are scheduled there. The cluster runtime on both nodes is containerd://2.3.6 (from kubectl get nodes -o wide), so no Docker daemon was required on the nodes.

- k8s/postgres.yaml and k8s/redis.yaml use ephemeral storage (no PVC) — the original postgres-data / redis-data 1Gi PVCs stayed Pending (0/2 nodes are available: pod has unbound immediate PersistentVolumeClaims) because the kubeadm cluster has no default StorageClass (kubectl get sc → none). The PVC blocks were removed; data lives in the container filesystem for this demo. This was documented as accepted data-loss tradeoff for the K8s demo (persistence was already proven in the Docker task).

- k8s/app.yaml env order is load-bearing: POSTGRES_PASSWORD (from postgres-secret) must come before DATABASE_URL=postgresql://barq_app:$(POSTGRES_PASSWORD)@postgres:5432/barq_tasks, otherwise kubelet leaves the literal $(...) and /ready returns 503 with dependency_error postgres OperationalError. Probes are liveness /health + readiness /ready.

- `k8s/app.yaml` uses image `docker.io/zinabelamir/barq-app:latest`. The cluster pulls it from Docker Hub, so the BONUS CD pipeline (`.github/workflows/cd.yml`) must have pushed it at least once, or `kubectl set image` must point at a tag you pushed.
- `k8s/secret.yaml` provides `POSTGRES_PASSWORD` for postgres and the app.
- `k8s/postgres.yaml` includes the `postgres-init` ConfigMap that creates the `records` table; `k8s/nginx.yaml` includes the `nginx-conf` ConfigMap and exposes NodePort `30080`.

Wait until everything is Ready, then confirm distribution across nodes:

```
kubectl -n barq-assessment get pods -o wide
kubectl -n barq-assessment rollout status deployment/barq-app --timeout=180s
kubectl -n barq-assessment rollout status deployment/postgres --timeout=120s
kubectl -n barq-assessment rollout status deployment/redis --timeout=120s
kubectl -n barq-assessment rollout status deployment/nginx --timeout=120s
```

```
[zinabelamir@k8s-master k8s]$ kubectl get nodes -o wide
kubectl get pods -A -o wide
kubectl cluster-info
NAME          STATUS   ROLES           AGE   VERSION    INTERNAL-IP      EXTERNAL-IP   OS-IMAGE                      KERNEL-VERSION                 CONTAINER-RUNTIME
k8s-master    Ready    control-plane   47h   v1.30.14   192.168.66.132   <none>        Rocky Linux 9.8 (Blue Onyx)   5.14.0-687.54.1.el9_8.x86_64   containerd://2.3.6
k8s-worker1   Ready    <none>          46h   v1.30.14   192.168.66.133   <none>        Rocky Linux 9.8 (Blue Onyx)   5.14.0-687.54.1.el9_8.x86_64   containerd://2.3.6
NAMESPACE         NAME                                       READY   STATUS    RESTARTS       AGE     IP               NODE          NOMINATED NODE   READINESS GATES
barq-assessment   barq-app-849774dcc6-blw7l                  1/1     Running   0              8m48s   10.244.1.13      k8s-worker1   <none>           <none>
barq-assessment   barq-app-849774dcc6-cw55h                  1/1     Running   0              8m46s   10.244.1.14      k8s-worker1   <none>           <none>
barq-assessment   barq-app-849774dcc6-d7wcb                  1/1     Running   0              8m44s   10.244.1.15      k8s-worker1   <none>           <none>
barq-assessment   nginx-7556dd5794-88qt5                     1/1     Running   0              35m     10.244.1.8       k8s-worker1   <none>           <none>
barq-assessment   nginx-7556dd5794-qfdrp                     1/1     Running   0              35m     10.244.1.7       k8s-worker1   <none>           <none>
barq-assessment   postgres-5c4ccfdf7c-wtt4l                  1/1     Running   0              29m     10.244.1.12      k8s-worker1   <none>           <none>
barq-assessment   redis-57d77659f7-mbt8m                     1/1     Running   0              37m     10.244.1.3       k8s-worker1   <none>           <none>
kube-flannel      kube-flannel-ds-r8qb2                      1/1     Running   4 (99m ago)    45h     192.168.66.133   k8s-worker1   <none>           <none>
kube-flannel      kube-flannel-ds-v9r4g                      1/1     Running   4 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
kube-system       coredns-55cb58b774-4wp9j                   1/1     Running   3 (100m ago)   47h     10.244.0.3       k8s-master    <none>           <none>
kube-system       coredns-55cb58b774-j4p5r                   1/1     Running   3 (100m ago)   47h     10.244.0.2       k8s-master    <none>           <none>
kube-system       etcd-k8s-master                            1/1     Running   3 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
kube-system       kube-apiserver-k8s-master                  1/1     Running   4 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
kube-system       kube-controller-manager-k8s-master         1/1     Running   3 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
kube-system       kube-proxy-bx7pl                           1/1     Running   3 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
kube-system       kube-proxy-kpn7k                           1/1     Running   5 (99m ago)    46h     192.168.66.133   k8s-worker1   <none>           <none>
kube-system       kube-scheduler-k8s-master                  1/1     Running   3 (100m ago)   47h     192.168.66.132   k8s-master    <none>           <none>
Kubernetes control plane is running at https://192.168.66.132:6443
CoreDNS is running at https://192.168.66.132:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy
```

9. Services & Endpoints Verification

```
kubectl -n barq-assessment get svc
kubectl -n barq-assessment get endpoints barq-app
kubectl -n barq-assessment get endpoints postgres
kubectl -n barq-assessment get endpoints redis
```

```
[zinabelamir@k8s-master barq-task]$ kubectl -n barq-assessment get svc
kubectl -n barq-assessment get endpoints barq-app
kubectl -n barq-assessment get endpoints postgres
kubectl -n barq-assessment get endpoints redis
NAME       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
barq-app   ClusterIP   10.96.221.88    <none>        8080/TCP       5h59m
nginx      NodePort    10.102.203.22   <none>        80:30080/TCP   5h59m
postgres   ClusterIP   10.98.10.141    <none>        5432/TCP       6h16m
redis      ClusterIP   10.99.105.169   <none>        6379/TCP       6h8m
NAME       ENDPOINTS                                              AGE
barq-app   10.244.1.10:8080,10.244.1.11:8080,10.244.1.12:8080   5h59m
NAME       ENDPOINTS          AGE
postgres   10.244.1.3:5432    6h16m
NAME    ENDPOINTS          AGE
redis   10.244.1.7:6379    6h8m
```

Expected services:

Service	Type	Port
`postgres`	ClusterIP	5432
`redis`	ClusterIP	6379
`barq-app`	ClusterIP	8080
`nginx`	NodePort	80 -> 30080

10. Connectivity & Response Testing

10.1 Test the app ClusterIP service from inside the cluster

```
kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/
```

Expected: JSON with `service: barq-api`, the configured `message`, and an `instance_id`.

```
[zinabelamir@k8s-master barq-task]$ kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Ne
ver -- \
  curl -s http://barq-app:8080/
{"instance_id":"barq-app-687c7d85b-94hff","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}
pod "temp-curl" deleted
[zinabelamir@k8s-master barq-task]$
```

10.2 Test health, readiness, records and counter

```
kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/health; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/ready; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"title":"k8s smoke test"}' http://barq-app:8080/records; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/counter; echo
```

`/ready` must report `"status": "ready"` with both `postgres` and `redis` ready.

```
[zinabelamir@k8s-master barq-task]$ kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/
{"instance_id":"barq-app-687c7d85b-94hff","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}
pod "temp-curl" deleted
[zinabelamir@k8s-master barq-task]$ kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/health; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/ready; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s -X POST -H 'Content-Type: application/json' \
  -d '{"title":"k8s smoke test"}' http://barq-app:8080/records; echo

kubectl -n barq-assessment run temp-curl --rm -it --image=curlimages/curl --restart=Never -- \
  curl -s http://barq-app:8080/counter; echo
{"instance_id":"barq-app-687c7d85b-94hff","service":"barq-api","status":"alive","version":"2.0.0"}
pod "temp-curl" deleted

{"dependencies":{"postgres":"ready","redis":"ready"},"instance_id":"barq-app-687c7d85b-p4h75","service":"barq-api","status":"ready","version":"2.0
.0"}
pod "temp-curl" deleted

{"instance_id":"barq-app-687c7d85b-75946","record":{"id":3,"title":"k8s smoke test"},"service":"barq-api","version":"2.0.0"}
pod "temp-curl" deleted

{"counter":1,"instance_id":"barq-app-687c7d85b-94hff","service":"barq-api","version":"2.0.0"}
pod "temp-curl" deleted

[zinabelamir@k8s-master barq-task]$
```

10.3 Test NGINX from outside the cluster (host)

```
curl -s http://192.168.66.132:30080/
curl -s http://192.168.66.133:30080/
```

Expected: the same app JSON, routed through NGINX (NodePort `30080`).

```
C:\Users\zinab>curl -s http://192.168.66.132:30080/
{"instance_id":"barq-app-687c7d85b-75946","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}

C:\Users\zinab>curl -s http://192.168.66.133:30080/
{"instance_id":"barq-app-687c7d85b-94hff","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}
```

10.4 Cross-node pod networking test (`network-test.yaml`)

`kubectl get nodes` proving Ready is not enough — you also want proof that a pod on one node can actually reach a pod on the other node (i.e. the Flannel overlay works). That is what the repo-root `network-test.yaml` is for. It creates two `busybox:1.36` pods that just sleep for an hour:

- `net-master`: pinned to `k8s-master` via `nodeSelector`, with a toleration for the

`node-role.kubernetes.io/control-plane:NoSchedule` taint (otherwise nothing would ever schedule on the master — see Section 7).

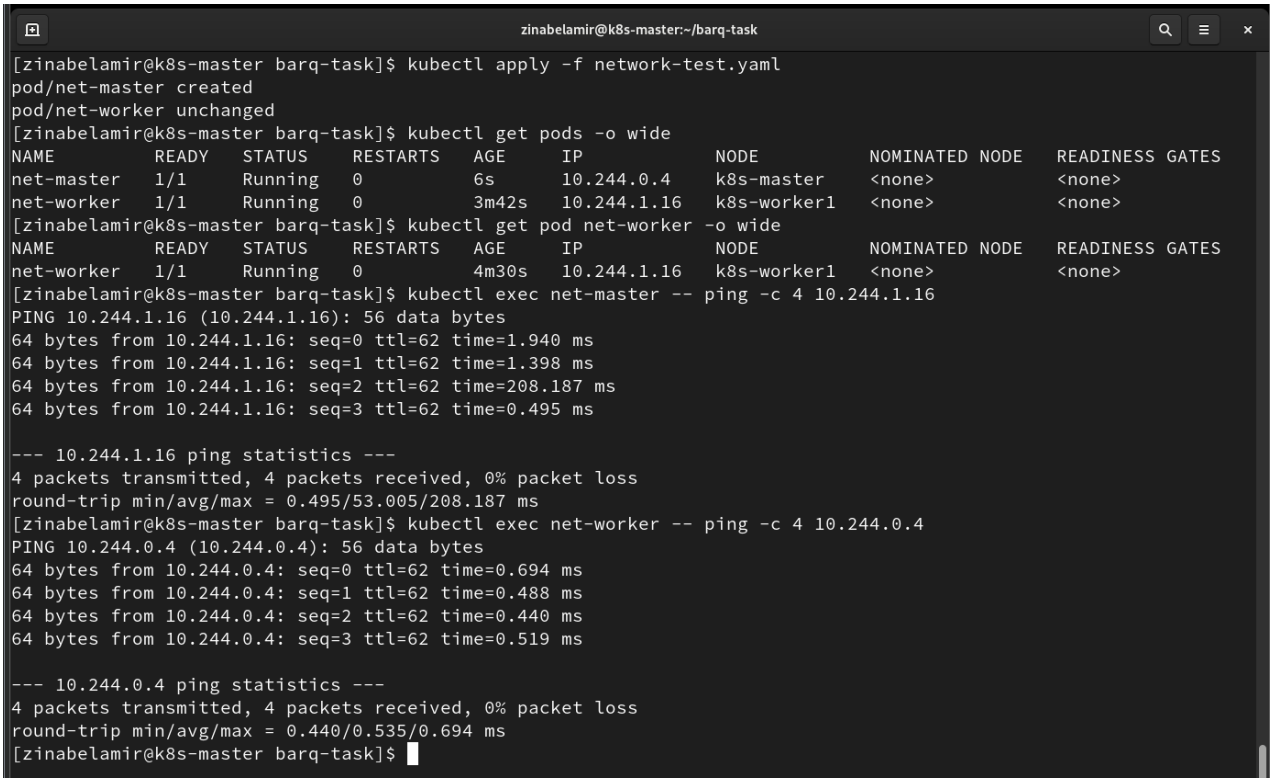
- `net-worker`: pinned to `k8s-worker1` via `nodeSelector` (no toleration needed).

Run the test (note: these pods land in the `default` namespace, not `barq-assessment`):

```
kubectl apply -f network-test.yaml
kubectl get pods -o wide
MASTER_POD_IP=$(kubectl get pod net-master -o jsonpath='{.status.podIP}')
WORKER_POD_IP=$(kubectl get pod net-worker -o jsonpath='{.status.podIP}')
echo "master=$MASTER_POD_IP worker=$WORKER_POD_IP"
kubectl exec net-master -- ping -c 3 $WORKER_POD_IP
```

```
kubectl exec net-worker -- ping -c 3 $MASTER_POD_IP
```

Expected: `0% packet loss` in **both** directions (master -> worker and worker -> master). That result proves cross-node pod-to-pod traffic flows through the CNI. Clean up afterwards so the test pods do not linger in the cluster:




```
kubectl delete -f network-test.yaml
kubectl get pods
```

11. BONUS: CI/CD Pipeline (Build, Push, Deploy with GitHub Actions)

11.1 How things worked BEFORE the bonus (manual flow)

Before the pipeline existed, every deployment was fully manual:

- The app Deployment used a **node-local image**: `image: localhost/barq-app:1.0` with

`imagePullPolicy: Never`. That image only existed on `k8s-master` itself, so no other node (and no fresh cluster) could pull it.

- There was **no container registry** in the flow. Updating the app meant logging into

the master over SSH and running each step by hand: `podman build`, `podman tag`, then `kubectl apply` / `kubectl set image`, then manually checking the rollout.

- There was **no automation or traceability**: nothing linked a git commit to the image

running in the cluster. If someone forgot a step, the cluster silently kept serving the old version, and there was no log proving which code was deployed.

11.2 Exact steps done for the bonus (with the actual commands)

Step 1 — Pointed the Deployment at a real registry (`k8s/app.yaml`, 2 lines).

```
- image: localhost/barq-app:1.0
- imagePullPolicy: Never
+ image: docker.io/zinabelamir/barq-app:latest
+ imagePullPolicy: IfNotPresent
```

Reason: with `localhost/...` + `Never`, the cluster could only ever run whatever was hand-built on that one machine. With `docker.io/...` + `IfNotPresent`, any node pulls the image the pipeline pushes (this exact change is what the pipeline's `kubectl apply` distributes on its first run).

Step 2 — Created the pipeline file `.github/workflows/cd.yml`.

Trigger (only relevant pushes start a deploy; anything else, e.g. `docs/`, is ignored):

```
on:
  push:
    branches: [main]
    paths:
      - "app/**"
      - "Dockerfile"
      - "requirements.txt"
      - "k8s/**"
      - ".github/workflows/cd.yml"
  workflow_dispatch:
```

Job (`runs-on: self-hosted`, i.e. it executes on `k8s-master` itself, where `podman` and `kubectl` with a working kubeconfig already exist). The steps, with the real commands exactly as executed:

```
# 1. checkout the code (actions/checkout@v4)
# 2. sanity check the runner tooling
whoami
podman --version
kubectl version --client
kubectl -n barq-assessment get nodes

# 3. compute tags from the commit SHA (SHORT_SHA = first 7 chars)
echo "SHORT_SHA=${GITHUB_SHA::7}" >> "$GITHUB_ENV"

# 4. log in to Docker Hub with the GitHub secrets
echo "${{ secrets.DOCKERHUB_TOKEN }}" | podman login docker.io \
  -u "${{ secrets.DOCKERHUB_USERNAME }}" --password-stdin

# 5. build BOTH tags from the repo Dockerfile
podman build \
  -t "docker.io/zinabelamir/barq-app:${SHORT_SHA}" \
  -t "docker.io/zinabelamir/barq-app:latest" \
  -f Dockerfile .

# 6. push BOTH tags to the registry
podman push "docker.io/zinabelamir/barq-app:${SHORT_SHA}"
```

```
podman push "docker.io/zinabelamir/barq-app:latest"

# 7. deploy: apply manifests, pin the exact SHA image, wait for the rollout
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/app.yaml
kubectl -n barq-assessment set image "deployment/barq-app" \
  "app=docker.io/zinabelamir/barq-app:${SHORT_SHA}"
kubectl -n barq-assessment rollout status "deployment/barq-app" --timeout=180s

# 8. verify (deploy-time evidence inside the run log itself)
kubectl -n barq-assessment get deployment barq-app \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
kubectl -n barq-assessment get pods -l app=barq-app -o wide
kubectl -n barq-assessment port-forward "deploy/barq-app" 18080:8080 &
curl -s http://127.0.0.1:18080/
curl -s http://127.0.0.1:18080/ready

# 9. clean the credentials at the end of every run
podman logout docker.io || true
```

Step 3 — Made the runner permanent on the VM. The first attempts used the manual `./run.sh`, which died on every `Ctrl-C` / closed terminal, so GitHub jobs sat at `Waiting for a runner to pick up this job...` forever. Fix (run once on `k8s-master` as the same user that ran `config.sh`):

```
cd ~/actions-runner
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

This registers the runner as a systemd service: it survives logout/terminal close and starts on boot. Verify on GitHub under `Settings -> Actions -> Runners` that `k8s-master` shows green `Idle`.

Step 4 — Added the registry credentials as GitHub secrets (never in code):

- Docker Hub: profile -> Security -> Personal Access Tokens -> Generate (read/write).
- GitHub repo `Settings -> Secrets and variables -> Actions`:

`DOCKERHUB_USERNAME = zinabelamir`, `DOCKERHUB_TOKEN = <the token>`.

Step 5 — Fixed the first red run. It failed at `kubectl apply` with `yaml: line 13: found a tab character that violates indentation` — line 13 of `k8s/app.yaml` had a hidden TAB (`[TAB]imagePullPolicy`). Fix: replaced it with 8 spaces like the neighboring lines, pushed again, run went green.

Committed + pushed, watched the new `cd` run go green, then confirmed on the cluster:

```
kubectl -n barq-assessment get deployment barq-app \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
# -> docker.io/zinabelamir/barq-app:<new-SHORT_SHA>
kubectl -n barq-assessment port-forward deploy/barq-app 18080:8080 &
curl -s http://127.0.0.1:18080/; echo
# -> "message": "Welcome to BARQ Systems"
```

```
[zinabelamir@k8s-master barq-task]$ curl -s http://127.0.0.1:18080/; echo
Handling connection for 18080
{"instance_id":"barq-app-687c7d85b-75946","message":"Welcome to BARQ Systems","service":"barq-api","version":"2.0.0"}

[zinabelamir@k8s-master barq-task]$
```

11.3 Result of the bonus

- Every relevant push to `main` now **automatically** builds a commit-tagged image,

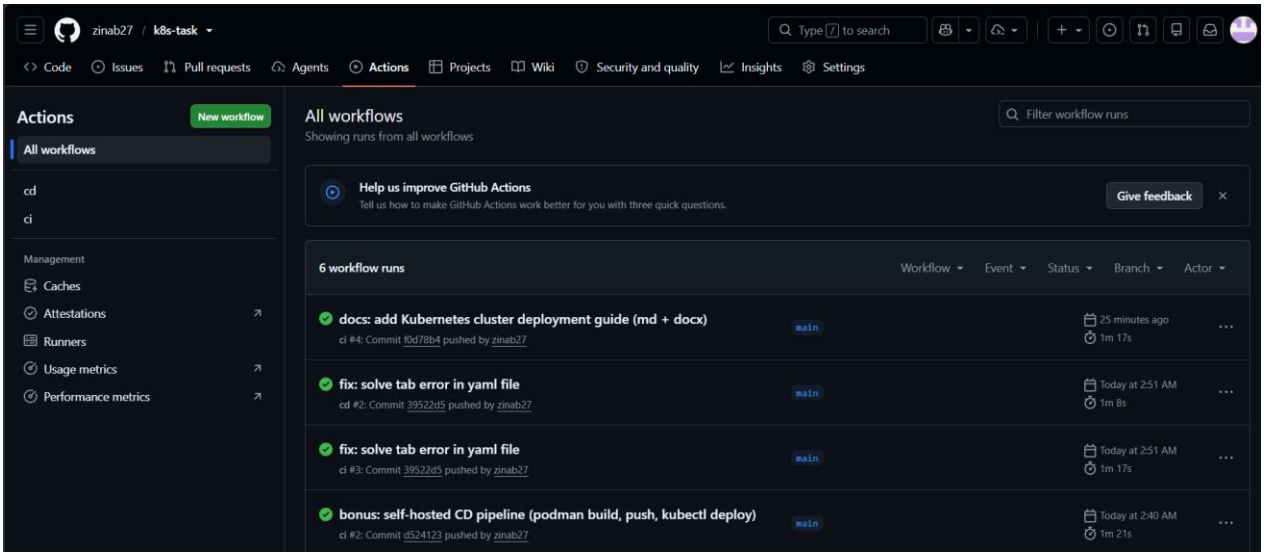
pushes it to Docker Hub (`:SHORT_SHA` + `:latest`), rolls it out to the `barq-app` Deployment, waits for the rollout, and curl-verifies the live app.

- No SSH or manual commands are needed for routine app updates.
- The evidence package is: the pipeline file (`.github/workflows/cd.yml`) + a green

`cd` run + `kubectl` output showing the new image tag + `curl /` output showing the changed message reaching the running application.

```
[zinabelamir@k8s-master barq-task]$ kubectl -n barq-assessment get deployment barq-app
-o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
docker.io/zinabelamir/barq-app:7df793e
[zinabelamir@k8s-master barq-task]$ curl -s http://127.0.0.1:18080/; echo
Handling connection for 18080
{"instance_id":"barq-app-54d9c6b459-4mffl","message":"Welcome to BARQ Systems - CD Test",
"service":"barq-api","version":"2.0.0"}

[zinabelamir@k8s-master barq-task]$
```



Troubleshooting Quick Reference

Symptom	Likely cause	Fix
Node stays `NotReady`	Flannel not applied or pod CIDR mismatch	Re-apply Flannel; confirm `--pod-network-cidr=10.244.0.0/16`
`ImagePullBackOff` on `barq-app`	Image not pushed to Docker Hub yet	Run the CD pipeline once (`Actions -> cd`), or `podman push docker.io/zinabelamir/barq-app:latest`
`CrashLoopBackOff` on `barq-app`	Missing secret or DB unreachable	`kubectl -n barq-assessment get secret postgres-secret`; check `/ready` output
Worker cannot join	Firewall or expired token	Stop firewalld for lab; regenerate join command with `kubeadm token create --print-join-command`
Pods only on master	Taint removed	Re-apply `kubectl taint nodes k8s-master node-role.kubernetes.io/control-plane:NoSchedule`